Warp

Warp 1.6.4 scans a repo for a plan, builds a schedule, and dispatches one Cursor subagent (a Shuttle) per ticket, each on its own branch and pull request. The parent agent is the only merger. You drive it from a Cursor agent window. This guide is the session, from install through stop, and where to look when something does not move. Every flag is in docs/COMMANDS.md and every config key in docs/CONFIG.md.

Where the files are

A local agent uses your clone. A cloud agent uses a Cursor virtual machine with its own clone. Warp commits `.warp/beam.json`, `.warp/journal.jsonl`, `.warp/STATUS.md` and `.warp/BOARD.md` on the base branch, so the next agent sees who is in flight, which locks are held, and which tickets are parked. Tokens, cost, API keys and webhook URLs are stripped before that commit. `.warp/config.yaml` stays gitignored.

Product code is a Shuttle pull request on `warp/<id>-<jira>` (`warp/<id>` when the ticket has no Jira key). A named run (`/warp-init <name>`) saves the same files, plus `stats.json` and a config copy without machine keys or secrets, to `.warp/runs/<name>/`, and its branches are `warp/<name>/<id>-<jira>`. `/warp-dashboard` writes `.warp/dashboard.html` locally; state commits leave it out. On a cloud VM, download what you want to keep before the VM is discarded.

Session, in order

Step	In the agent window	What you should see
Install	Import the plugin, install Warp from Customize, reload. Then `/warp-init` (or `/warp-init <name>` for a named run).	`/warp-scan` and `/warp-start` in the command list. `.warp/config.yaml` written; Jira and Slack tools allowed.
Plan	A plan file in the repo, or, when the work is already in Jira, `/warp-from-jira version:1.5.5`.	`next: /warp-scan warp-plans/version-1.5.5`, with a warning for each issue missing a size, locks, or a blocker.
Scan	`/warp-scan` or `/warp-scan <folder>`	Ticket count, format, gate order, `12 tickets: 9 keyed, 3 need mapping`, run stopped.
Start	`/warp-start`, or a range: `/warp-start G0-G2`	`run: was stopped. Starting.`, `unfinished:`, `parent: session <id>`. Shuttles claim up to `maxAgents`.
Watch	`/warp-status`, `/warp-dashboard`	Open tickets with alarms first; the swimlane, board and table.
Pause	`/warp-pause`	Every agent ended, branches and pull requests kept. `halt: paused agents=<n>`.
Edit	`/warp-export`, edit `WARP_PLAN.json`, `/warp-import <path>`	Plan replaced, status kept for ids that remain, run stopped.
Carry on	`/warp-start` or `/warp-resume` (the same command)	Unfinished work first: fixes, halted Shuttles, then new tickets.
Stop	`/warp-stop`	Same teardown as pause, plus the completion report and `stoppedAt`.

Where the plan comes from

`/warp-scan` reads, richest first: `WARP_PLAN.json`; a `schedule.json` (Warp's own shape, or build-schedule/2, the shared FORGEQA/HOS format); `CURSOR_PLAN.md`; a Jira JSON export; a markdown table with `id` and `deps`. Warp does not invent tickets.

- No plan file, work in Jira: `/warp-from-jira <label>[,<label>]` reads the issues with those labels (`--match all` for every label) and writes `warp-plans/<labels>/plan/schedule.json`, `FROM_JIRA.md`, and the Jira keys. Done issues and epics are left out. It writes nothing to Jira and refuses a folder that already has a plan unless `--force`.
- A build-schedule/2 file keeps its planned slots, priority gates, merge checks and lanes; the dashboard draws it as written. A schedule that spans several repos is split by repo (config `repo`, `repoRuns`, `externalRepos`).
- `python3 <plugin>/scripts/buildsched.py show <schedule.json> --repo <name>` prints what Warp will read, without scanning.

Run control

`/warp-start` and `/warp-resume` are one command: from a stopped, paused or running run, it sets the run running and says which. `/warp-pause` and `/warp-stop` are one teardown; stop also writes the report. A named run that is live on another machine needs `--takeover`.

Cap	Counts	Default
`maxAgents`	Agents running at once: every Shuttle, whatever its step (implement, fix, rebase, rerun, repair)	18
`maxInProgress`	Tickets open at once: started and not merged or parked. Holds only new tickets	20

A ticket is ready when its blockers are merged and no lock overlaps a ticket in flight; with gates or other runs, also the block order, `paths`, other runs' locks, and `run:id` blockers in another run. Dispatch order: a base-branch fix, then the ★ critical chain, then a priority gate's path, then priority, then rank.

Gates and blocks

- `gateOrder` strict (the default for most plans): block G3 starts when every earlier block is cleared. `/warp-start G0-G3`, `G0-`, `-G2` and `G5` run a range; a named gate works too: `G-RBAC`, `G0..G-RBAC`.
- A build-schedule/2 schedule runs in deps order with milestone gates unless its `model.gate_order` is strict: a pending gate holds nothing, a red blocking gate stops every ticket outside it, a red non-blocking gate stops the tickets that need its members.
- A `gateOrder` line in `.warp/config.yaml` overrides the schedule. Configs written by 1.6.0–1.6.2 carry `gateOrder: strict`; delete it for milestone gates. `/warp-scan` prints the order that will apply.
- When a gate turns green, Warp prints its `on_green` command for the parent to run and report, and the programs it `starts`.

Several runs, several repos

`/warp-init <name>` names a run (`core`, `admin`, `ui`). Each run has its own parent, beam and branches, and publishes `.warp/runs/<name>/` to the base branch. `startAfter: core:G2` waits for another run's gate (`--now` overrides). A blocker written `core:P-014` waits for that run's ticket. `paths` limits what a run may change, and `baseFixOwner` names the one run that fixes a red base branch. Across repos, `repoRuns` names the run that works the other repo and `externalRepos` says where to fetch its published beam. `/warp-runs` lists the runs; `/warp-delete <name> --apply` removes one.

Export, edit, import

Pause first. `/warp-export` writes `.warp/WARP_PLAN.json` and `.warp/WARP_PLAN.md`. Edit the JSON: `deps`, `locks`, `size`, `autoMerge`, `critical`, and gate `members` and `checks`. Keep an id stable to keep its pull request, tokens and

minutes. Then `/warp-import <path>`: the graph is replaced, status is copied onto ids that remain, removed ids are dropped, the run is stopped, and config is not touched.

File	Edit	Role
`.warp/WARP_PLAN.json`	Yes	The file import reads.
`.warp/WARP_PLAN.md`	Read	Analysis brief for another model. Not imported.
`.warp/config.yaml`	Yes	Model, caps, messenger, Jira. Local; survives import.
`.warp/beam.json`	No	Live status. Scripts write it; committed on the base branch.
`.warp/STATUS.md`, `BOARD.md`	No	Done, working, left. `/warp-status` rewrites them.
`.warp/journal.jsonl`	No	Append-only log, committed with tokens stripped.
`.warp/agents.json`	No	Every agent spawned and ended. `/warp-list` reads it.
`.warp/dashboard.html`	No	The run dashboard. Local; Reload in the page re-reads it.

Configuration

One file, `.warp/config.yaml`. `/warp-init` writes it and adds new keys on later runs without touching your values. Change it, then restart, so the next tick re-reads it. The main keys:

Key	Default	Meaning
`model`	`claude-sonnet-5-5-high`	Coding model slug. Must match the Cursor model picker.
`modelS` … `modelXL`	empty	The Shuttle model for that ticket size. Empty uses `model`.
`fixModel`, `fixModelAfter`	empty, 2	A stronger model for a ticket's second and later fix rounds. Empty keeps the ticket's model.
`maxAgents` / `maxInProgress`	18 / 20	The two caps above. There is no per-person cap.
`autoMergeSizes`	S, M	Sizes that merge after Bugbot and CI without a human approval.
`runner` / `launch`	cloud / worktree	Where the parent runs; one git worktree per ticket.
`gateOrder`	unset	strict, except a build-schedule/2 schedule (deps). A line here overrides both.
`run`, `scanFolder`	none, empty	This checkout's run name; the folder the last scan read.
`startAfter`, `paths`, `baseFixOwner`	empty	Cross-run waits, the paths this run may change, who fixes a red base.
`repo`, `repoRuns`, `externalRepos`	origin, {}, {}	Multi-repo schedules: this repo, the run per other repo, where to fetch it.
`jiraProject`, `jiraMcp`	empty, atlassian	Jira project key and server. Scan fills the project when one is clear.
`jiraInProgressStatus` … `jiraDoneStatus`	In Progress, In Review, QA Ready, Done	The four Jira moves: claim, review, approval, merge.
`bugbotRequired`, `maxFixAttempts`	true, 5	No merge without Bugbot; a ticket parks after five fixes.
`checkCommand`	empty	Run on the rebased head before merge, for example `make ci`.

Key	Default	Meaning
`messenger`, `notify`	both, verbose	Slack, Teams or both; `quiet` posts only alarms, waits and red gates.

Status, dashboard and help

`/warp-status` lists every ticket that is not merged, alarms and stalls first. `/warp-dashboard` writes one HTML page: the plan as a slot swimlane (Plan and Actual clocks), a board by gate block, status, module, lane or repo, a sortable table with the four pull request timestamps, and totals for run estimate, agent hours, people hours and tickets. It also opens a `schedule.json` before any run. In Slack or Teams, `warp:status`, `warp:pause admin` and `warp:start admin G2-G3` address a run by name.

When something does not move, start with docs/RUNBOOK.md. The Jira path is docs/GUIDE.md, named runs docs/RUNS.md, agent start and teardown docs/LIFECYCLE.md, and every change CHANGELOG.md.